RELATÓRIO DE CONTINUIDADE

PreJud — Integração do Dashboard do Freelancer

Documento de sessão em HTML para salvar em PDF, consolidando o que foi feito, onde parou e como continuar exatamente do ponto atual, sem quebrar a base existente.

STATUS ATUAL	FASE REAL	RISCO ATUAL	PRÓXIMO FOCO
Dashboard abriu com dados	Integração inicial concluída	Baixo, com atenção ao manual	Teste funcional do fluxo

1. Objetivo desta seção

O objetivo da seção foi integrar o dashboard do freelancer ao Firestore com o menor risco possível, preservando a UI existente, sem refatorar arquitetura global e sem mexer no que já estava funcionando no projeto. A abordagem adotada foi de adição incremental: criar um service novo e um hook novo para alimentar o dashboard, em vez de reescrever services antigos.

> Regra seguida: **adicionar, não substituir**. Isso respeita a orientação do manual de manter o domínio centralizado, evitar lógica paralela em componentes e não quebrar fluxos já estáveis.

2. O que foi feito nesta sessão

1. Foi analisado o contexto do projeto e a necessidade de integrar o dashboard do freelancer sem mexer no núcleo existente.
2. Foi identificado que o campo real salvo no Firestore para o criador do acordo é **freelancerId**, enquanto uma função antiga do service ainda buscava por **creditorId/debtorId**.
3. Para evitar risco de regressão, foi decidido **não alterar o service antigo** e criar um arquivo novo dedicado ao dashboard.

4. Foi criado o arquivo **services/dashboardService.ts**.
5. Foi criado o arquivo **hooks/useFreelancerDashboard.ts**.
6. O arquivo **app/(dashboard)/dashboard/page.tsx** foi ajustado para consumir dados reais via hook novo.
7. Foram corrigidos erros de TypeScript: import ausente do hook, tipos implícitos *any*, e ajustes de casting no service.
8. A checagem com **npx tsc --noEmit** terminou sem erros.
9. Ao abrir o dashboard, surgiu erro de índice composto do Firestore para a collection **notices**.
10. O índice foi criado e, após isso, o dashboard passou a abrir normalmente.

3. Resultado atual

Concluído

- Dashboard carrega sem erro de TypeScript.
- Hook novo criado e reconhecido pelo projeto.
- Service novo criado sem tocar no fluxo antigo.
- Índice do Firestore criado para notices.
- Página do dashboard abriu com sucesso.

4. Arquivos criados ou alterados

Arquivo	Ação	Objetivo	Risco
services/dashboardService.ts	Criado	Buscar agreements, charges e notices do freelancer e consolidar stats	Baixo
hooks/useFreelancerDashboard.ts	Criado	Centralizar carregamento, loading, error e refresh do dashboard	Baixo
app/(dashboard)/dashboard/page.tsx	Alterado	Trocar mocks do dashboard por dados reais do Firestore	Médio

5. Diagnóstico técnico consolidado

5.1 Descobertas importantes

- O Firestore está salvando **freelancerId**, e não **creditorId**.
- A função antiga **getAgreementsByUser** ficou inconsistente com os tipos atuais e não foi usada na integração nova.
- O dashboard original estava fortemente baseado em dados mockados.
- A integração nova preservou a estrutura visual e substituiu apenas a fonte de dados.

5.2 O que não foi mexido

- State machine principal.
- Permissões do domínio.
- Services antigos do fluxo central.
- Arquitetura global do projeto.
- Collections existentes no Firestore.

6. Aderência ao manual técnico

O manual reforça que a entidade central é o **Agreement**, que a UI deve consumir regras vindas de hooks/services e que não se deve criar lógica de domínio diretamente em componentes. Também orienta a ordem de evolução: domínio, frontend bilateral, integração com Firestore e depois reputação. O trabalho executado nesta sessão respeitou esse princípio ao isolar a integração em hook + service e evitar reescrita do domínio central. O manual também alerta para não criar lógica paralela em múltiplos lugares e não bypassar a timeline. □filecite□turn0file0□

Ponto do manual	Aplicação na sessão
Pages/UI → Hooks → Services → Lib/Types	O dashboard passou a consumir **useFreelancerDashboard**, que usa **dashboardService**, sem jogar regra de dados no JSX. □filecite□turn0file0□
Agreement como entidade central	A consulta base do dashboard foi construída a partir da collection agreements, e charges/notices foram agregadas a partir dos agreementIds. □filecite□turn0file0□

Ponto do manual	Aplicação na sessão
Nunca criar lógica de domínio diretamente em componentes	Foi preservado o componente de página e a integração foi empurrada para hook/service. □filecite□turn0file0□
Evitar duplicação e lógica paralela	Não foi alterado o fluxo antigo; foi criado um módulo isolado para o dashboard, reduzindo risco imediato, mas isso deve ser revisado depois para não virar duplicação permanente. □filecite□turn0file0□

7. Onde exatamente parou

A sessão parou no momento em que:

- o índice composto do Firestore para **notices** foi criado;
- o dashboard abriu com sucesso;
- foi perguntado **“o que testo?”**;
- o próximo passo natural passou a ser **teste funcional do dashboard com dados reais**.

Ponto atual Não é hora de refatorar. É hora de validar se a integração realmente está mostrando dados certos e se reage à criação de novos acordos.

8. O que já está validado

- OK Arquivos novos criados.
- OK Imports corrigidos.
- OK TypeScript sem erros.
- OK Índice do Firestore criado.
- OK Dashboard abriu.

O que ainda **não** foi validado é se os números, listas e histórico estão coerentes com os documentos reais do Firestore.

9. Continuidade exata a partir daqui

A continuidade correta, seguindo o manual e sem pular etapa, é de **teste funcional**. A sequência abaixo mantém o trabalho seguro:

Passo 1 — validar leitura do dashboard

- Verificar se os cards de stats exibem números reais ou zero coerente.
- Verificar se a lista “Meus acordos” mostra documentos do freelancer logado.
- Verificar se “Cobranças recentes” e “Histórico recente” aparecem sem erro.

Passo 2 — validar reatividade mínima

- Criar um novo acordo pelo fluxo já existente.
- Voltar ao dashboard.
- Confirmar se o acordo novo aparece na lista.

Passo 3 — validar consistência com Firestore

- Conferir na collection agreements se o documento possui **freelancerId** do usuário logado.
- Conferir o status salvo.
- Conferir se createdAt, updatedAt e deadline estão vindo corretamente.

Passo 4 — somente depois disso

- Ajustar mapeamentos finos de status.
- Melhorar cliente das cobranças, hoje ainda simplificado.
- Decidir se ativa realtime ou mantém refresh manual.

10. Próximos comandos recomendados

Como a condução está sendo feita em modo seguro e um comando por vez, a continuidade recomendada é:

Comando 1 — subir o ambiente, se ainda não estiver rodando

```
cd "C:\prejud-saas-new\frontend"
npm run dev
```

Comando 2 — abrir a página do dashboard no navegador

```
http://localhost:3000/dashboard
```

Comando 3 — conferir logs do navegador

```
Esperado no console do browser:
[DashboardService] Buscando agreements para: [uid]
[DashboardService] Encontrados X agreements
```

Comando 4 — se precisar inspecionar o dashboard aberto no código

```
code "C:\prejud-saas-new\frontend\app\(dashboard)\dashboard\page.tsx"
```

11. Testes objetivos que devem ser feitos agora

Teste	Como validar	Resultado esperado
Stats do dashboard	Abrir o dashboard e comparar com dados do Firestore	Números coerentes ou zero real
Lista de acordos	Verificar se acordos exibidos pertencem ao freelancer logado	Somente acordos do próprio usuário
Novo acordo	Criar um acordo e voltar ao dashboard	Item novo aparece na listagem
Cobranças recentes	Se existirem charges, validar exibição	Sem erro e com datas formatadas
Histórico recente	Se existirem notices, validar exibição	Sem erro de index e sem quebra visual

12. Pendências reais

- O campo de cliente nas cobranças ainda está simplificado.
- O mapeamento de status do backend para a UI ainda precisa ser revisado com mais calma.
- Falta validar comportamento com base real mais ampla, não apenas abrir a página.
- Falta decidir se o dashboard usará realtime ou refresh manual como padrão.

Importante Ainda não é momento de mexer em state machine, permissões ou timeline sem fechar essa validação da leitura básica.

13. Riscos e cuidados para a próxima sessão

- Não substituir services antigos agora.
- Não mexer em collections, nomes de campos ou rotas sem necessidade.
- Não colocar regra de negócio dentro do page.tsx.
- Não pular de dashboard para reputação, automação ou auditoria antes de estabilizar a leitura de dados.

> O manual recomenda que cada fase esteja estável antes de iniciar a próxima. Na prática, isso significa: primeiro provar que o dashboard lê corretamente; só depois evoluir ação, realtime, cliente bilateral e outros módulos. □filecite□turn0file0□

14. Resumo executivo

A integração inicial do dashboard do freelancer foi concluída com abordagem conservadora e sem refatoração agressiva. O projeto passou por criação de service/hook dedicados, correção de erros de TypeScript, criação do índice composto do Firestore e abertura bem-sucedida do dashboard. O trabalho parou no ponto certo: antes de novas mudanças, deve-se testar visualmente e funcionalmente a consistência dos dados exibidos.

Em outras palavras: **esta sessão terminou em um ponto bom**. O próximo passo não é codar mais; é validar o que já entrou.

Relatório gerado para continuidade segura do projeto PreJud.